



UNIVERSIDAD AUTÓNOMA DEL ESTADO DE MÉXICO

FACULTAD DE INGENIERÍA

Proyecto Integral de Ingeniería de Software

Documentación del Sprint 2

Casos de uso implementados · Fase RUP: Elaboración

Sistema Inteligente para la Gestión y Priorización de Incidencias de Soporte Técnico (SIGPI)

Derivado del Tablero Kanban v2 y de las Historias de Usuario v3 · FastAPI en capas · React + TypeScript

EQUIPO

Haide Sánchez Gutiérrez · Laura Valeria Rodríguez Morales

Jesús Iniesta Valverde · Emmanuel Bernal Espejel

DOCENTE

Silvia Edith Albarrán Trujillo

METODOLOGÍA

Modelo de proceso: RUP (Proceso Unificado) | Seguimiento: Scrum

Toluca de Lerdo, Estado de México · 4 de octubre de 2026

Contenido

1. Objetivo y alcance del Sprint 2 **3**

2. Sprint Backlog y estado del tablero Kanban **4**

3. Modelo de casos de uso del sprint **5**

4. Arquitectura aplicada en los casos de uso **6**

5. Casos de uso realizados **8**

CU-S2-01 · Administrar cuentas y roles (HU-10.1) 8

CU-S2-02 · Mantener el catálogo de técnicos y áreas (HU-11.1) 12

CU-S2-03 · Registrar una solicitud con folio (HU-01.1) 16

CU-S2-04 · Registrar el avance en la bitácora: diseño (HU-04.1) 20

CU-S2-05 · Auditar las acciones de administración (HU-10.3) 22

CU-S2-06 · Definir y versionar los acuerdos de nivel de servicio (HU-08.1) 25

6. Base ejecutable: migraciones, integración continua y contenedores **30**

7. Pruebas y calidad **33**

8. Trazabilidad: historia → tarjeta → código → prueba **36**

9. Velocidad del sprint **37**

10. Observaciones y ajustes para el Sprint 3 **38**

Referencias **39**

1. Objetivo y alcance del Sprint 2

El Sprint 2 corresponde a la fase de **Elaboración** del Proceso Unificado. Su objetivo, fijado en el Sprint Backlog 2 (tarjeta S2-18), fue construir la **base ejecutable** del sistema (modelos y migraciones, integración continua, Docker Compose y cliente API tipado) y entregar en todas sus capas las historias que sirven de cimiento al resto del producto: cuentas y roles (HU-10.1), auditoría (HU-10.3), catálogo de técnicos y áreas (HU-11.1) y acuerdos de nivel de servicio (HU-08.1). Además se implementó el servicio de alta de solicitudes (HU-01.1) y se diseñó la bitácora de atención (HU-04.1).

Este documento describe los **seis casos de uso trabajados** durante el sprint. Para cada uno presenta su ficha (actor, precondiciones, flujos y postcondiciones), su relación con las historias de usuario y las reglas de negocio, la implementación por capas con fragmentos del código fuente, capturas de las pantallas y las pruebas automatizadas que verifican los criterios de aceptación. Al final se registran la trazabilidad, la velocidad del sprint y los ajustes para el Sprint 3.

6 / 6
Historias comprometidas con sus tarjetas del sprint entregadas

17 / 17
Tarjetas de desarrollo entregadas (144 h)

95
Pruebas de backend aprobadas (pytest)

16
Pruebas de frontend aprobadas (Vitest + Testing Library)

Dato	Valor
Fase RUP	Elaboración (Sprint 2 de 5)
Capacidad planeada	154 h (144 h de desarrollo y 10 h de gestión), ≈ 19.2 h por integrante por semana
Periodo con actividad en el repositorio	30 de septiembre al 4 de octubre de 2026 (commits y *pull requests* #2 a #8 etiquetados con `S2-xx`)
Corte de este documento	4 de octubre de 2026, rama `main` en el commit `92674f9`
Roles Scrum	Product Owner: Haide Sánchez (ya no forma parte del equipo; rol por reasignar) · Scrum Master: Emmanuel Bernal · Developers: todo el equipo
Fuentes	Tablero Kanban SIGPI v2 · Historias de Usuario SIGPI v3 · Sprint Backlog 2, RiskLog v2 y Registro de velocidad del Sprint 2 (carpeta `backend/docs/Sprint_02V1`) · repositorio del proyecto

Tabla 1. Datos generales del Sprint 2.

1.1 Historias de usuario comprometidas

Historia	Título	Actor	Prioridad · tamaño	Tarjetas S2	Caso de uso	Resultado
HU-10.1	Administrar cuentas y roles	ACT-09	Debe · M	S2-04, S2-05, S2-06	CU-S2-01	Realizada
HU-11.1	Mantener el catálogo de técnicos y áreas	ACT-09	Debe · M	S2-10, S2-11, S2-12	CU-S2-02	Realizada
HU-01.1	Registrar una solicitud desde el portal	ACT-01	Debe · M	S2-16	CU-S2-03	Parte S2 realizada — continúa en S3
HU-04.1	Registrar el avance en una bitácora única	ACT-02 · ACT-03	Debe · M	S2-17	CU-S2-04	Diseño realizado — servicio en S3
HU-10.3	Auditar las acciones de administración	ACT-09	Debe · S	S2-07, S2-08, S2-09	CU-S2-05	Realizada
HU-08.1	Definir y versionar los acuerdos de nivel de servicio	ACT-04	Debe · M	S2-13, S2-14, S2-15	CU-S2-06	Realizada

Tabla 2. Historias del Sprint 2, caso de uso que las documenta y resultado. Las historias HU-01.1 y HU-04.1 tienen tarjetas en sprints posteriores (Tablero Kanban v2, tabla 11).

2. Sprint Backlog y estado del tablero Kanban

Una tarjeta se considera entregada cuando su producto está integrado en la rama `main` y, en el caso de las tarjetas de servidor e interfaz, cuenta con pruebas automatizadas (Definición de Terminado, Historias de Usuario v3, sección 2.6). Cada tarjeta indica su responsable en el tablero y, cuando fue distinto, quién la ejecutó; el detalle por commit está en el Registro de velocidad del Sprint 2.



Figura A. Estado del tablero Kanban del Sprint 2 al 4 de octubre de 2026. Borde azul: desarrollo; borde ámbar: gestión.

2.1 Tarjetas de gestión

Tarjeta	Descripción	Entregable (en backend/docs/Sprint_02V1/)	Estado
S2-18	Planeación y refinamiento	`Sprint_Backlog_2_SIGPI.pdf`	En validación
S2-19	Revisión del catálogo de riesgos	`RiskLog_v2_SIGPI.pdf`	Entregada
S2-20	Sprint Review 2 con el DTIC-FI	Este documento	Al cierre
S2-21	Retrospectiva 2 y velocidad	`Registro_de_Velocidad_Sprint_2_SIGPI.pdf`	En validación

Tabla 3. Tarjetas de gestión del Sprint 2 y su entregable en el repositorio.

3. Modelo de casos de uso del sprint

Los casos de uso del sprint provienen de los casos de uso del sistema del Modelado de Negocios (SCU-01, SCU-04, SCU-08, SCU-10 y SCU-11). Cada historia de usuario concreta un escenario de uno de ellos. El diagrama distingue los casos implementados (borde verde) del que en este sprint solo se diseñó (borde azul punteado).

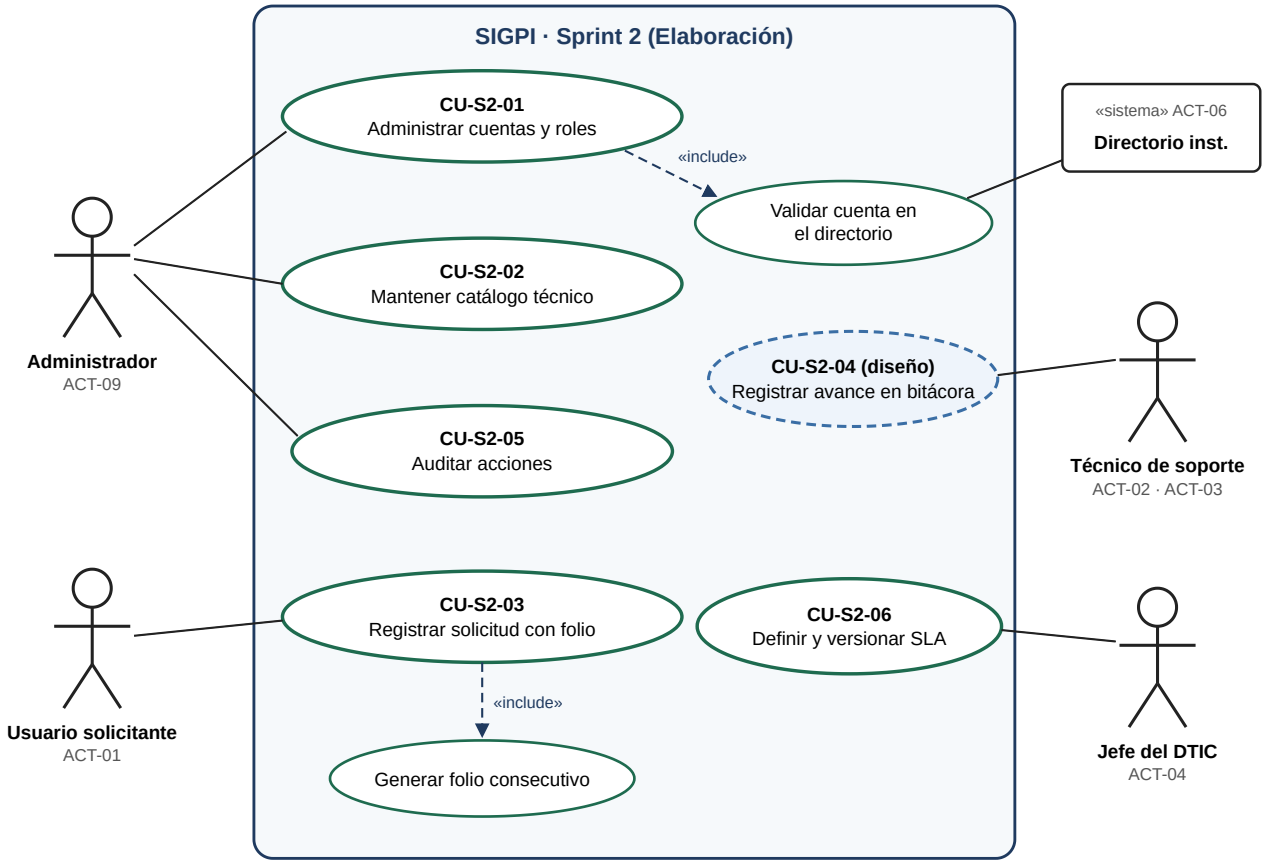


Figura 1. Diagrama de casos de uso del Sprint 2 (UML). Elaboración propia.

Caso de uso	Nombre	Caso del sistema	Historia	Reglas	Estado
CU-S2-01	Administrar cuentas y roles	SCU-10	HU-10.1	RN-19	Implementado
CU-S2-02	Mantener el catálogo de técnicos y áreas	SCU-11	HU-11.1	RN-08	Implementado
CU-S2-03	Registrar una solicitud con folio	SCU-01	HU-01.1	RN-01, RN-02, RN-03	Implementado (servidor)
CU-S2-04	Registrar el avance en la bitácora	SCU-04	HU-04.1	RN-10	Diseñado
CU-S2-05	Auditar las acciones de administración	SCU-10	HU-10.3	RN-19	Implementado (servidor)
CU-S2-06	Definir y versionar los acuerdos de nivel de servicio	SCU-08	HU-08.1	RN-05	Implementado

Tabla 4. Casos de uso del Sprint 2 y su origen en el Modelado de Negocios.

4. Arquitectura aplicada en los casos de uso

Todos los casos de uso siguen la arquitectura en capas definida en el ADR del Sprint 1 (S1-03). La regla de dependencia es única: cada capa conoce solo a la inmediatamente inferior. Las reglas de negocio viven en los servicios y en el dominio, de modo que se pueden probar sin servidor web y con una base de datos en memoria.

Capa	Paquete	Responsabilidad en el Sprint 2
Presentación (API)	`api/usuarios.py`, `api/routers/catalog.py`, `solicitudes.py`, `sla.py`	Recibe la petición HTTP, valida el cuerpo con Pydantic y traduce las excepciones del servicio a códigos HTTP (404, 409, 422).
Esquemas	`schemas/identity.py`, `catalog.py`, `request.py`, `sla.py`	Contratos de entrada y salida; rechazan campos desconocidos y valores inconsistentes.
Aplicación (servicios)	`services/usuario_service.py`, `catalog_service.py`, `solicitud_service.py`, `auditoria_service.py`, `sla_service.py`	Orquestan el caso de uso y aplican las reglas RN-xx.
Dominio	`domain/directory.py`, `domain/priority.py`	Puerto del directorio institucional y matriz de prioridad, sin dependencias de framework.
Acceso a datos	`repositories/*`, `models/*`, `alembic/`	Consultas SQLAlchemy asíncronas y migraciones versionadas, incluido el disparador de la auditoría.
Integración	`integrations/directory_client.py`	Adaptador provisional del directorio; el de Microsoft 365 llega en S3-03 sin cambiar las demás capas.
Frontend	`features/users`, `catalog`, `sla`; `api/*.ts`	Páginas React + TypeScript con TanStack Query; tipos generados desde el esquema OpenAPI.

Tabla 5. Capas que intervienen en los casos de uso del Sprint 2.

La capa de dominio declara el **puerto** del directorio institucional como un protocolo. El servicio de cuentas depende de esta abstracción y no del proveedor concreto, lo que permite usar un adaptador de prueba hoy y el de Microsoft 365 en el Sprint 3.

backend/app/domain/directory.py — líneas 13–24

```python
@dataclass(frozen=True)
class DirectoryProfile:
    institutional_id: str
    full_name: str
    email: str
    user_type: UserType


class DirectoryClient(Protocol):
    async def fetch_profile(self, institutional_id: str) -> DirectoryProfile | None:
        """Return the profile for an institutional account, or None if unknown."""
        ...
```

Figura 2. *Puerto del directorio institucional en la capa de dominio (S2-04).*

4.1 Cliente API tipado desde OpenAPI (S2-03)

FastAPI publica el contrato de la API en `/openapi.json`. La herramienta *openapi-typescript* genera a partir de él el archivo `frontend/src/api/openapi.ts` (≈ 1 350 líneas), y `types.ts` expone los tipos que usan las pantallas. Así, si un esquema de Pydantic cambia, el compilador de TypeScript señala los componentes afectados y el contrato no se duplica a mano.

frontend/src/api/types.ts — líneas 1–28

```ts
import type { components } from './openapi'

export type ApiSchemas = components['schemas']
export type UserRole = ApiSchemas['RoleName']
export type User = Omit<ApiSchemas['UserRead'], 'roles'> & {
  roles: UserRole[]
}
export type CreateUserInput = ApiSchemas['UserCreate']
export type UpdateUserInput = ApiSchemas['UserUpdate']
export type ServiceArea = ApiSchemas['ServiceAreaRead']
export type OrganizationalUnit = ApiSchemas['OrganizationalUnitRead']
export type Category = ApiSchemas['CategoryRead']
export type Specialty = ApiSchemas['SpecialtyRead']
export type Shift = ApiSchemas['ShiftRead']
export type Technician = ApiSchemas['TechnicianRead']
export type TechnicianUpdate = ApiSchemas['TechnicianProfileUpdate']
export type ServiceAreaInput = ApiSchemas['ServiceAreaCreate']
export type CategoryInput = ApiSchemas['CategoryCreate']
export type SpecialtyInput = ApiSchemas['SpecialtyCreate']
export type ShiftInput = ApiSchemas['ShiftCreate']
export type Request = ApiSchemas['RequestRead']
export type CreateRequestInput = ApiSchemas['RequestCreate']
export type Priority = ApiSchemas['Priority']
export type SlaAgreement = ApiSchemas['SlaAgreementRead']
export type CreateSlaAgreementInput = ApiSchemas['SlaAgreementCreate']
export type SlaVersion = ApiSchemas['SlaVersionRead']
export type CreateSlaVersionInput = ApiSchemas['SlaVersionCreate']
export type EffectiveSla = ApiSchemas['EffectiveSlaRead']
```

Figura 3. *Tipos del frontend derivados del esquema OpenAPI generado (S2-03).*

frontend/src/api/client.ts — líneas 29–56

```ts
export async function apiFetch<T>(
  path: string,
  init?: RequestInit,
): Promise<T> {
  const headers = new Headers(init?.headers)

  if (init?.body && !headers.has('Content-Type')) {
    headers.set('Content-Type', 'application/json')
  }

  const response = await fetch(`${apiUrl}${path}`, {
    ...init,
    headers,
  })

  if (!response.ok) {
    let detail = `API request failed with status ${response.status}`
    try {
      const body = (await response.json()) as { detail?: unknown }
      detail = formatApiDetail(body.detail) ?? detail
    } catch {
      // Keep the status-based message when the server response is not JSON.
    }
    throw new Error(detail)
  }

  return response.json() as Promise<T>
}
```

Figura 4. *Cliente HTTP común: convierte el campo detail de FastAPI en un mensaje para la interfaz (S2-03).*

5. Casos de uso realizados

Cada caso de uso se documenta con su ficha, los criterios de aceptación de la historia de usuario y su estado de cumplimiento, la implementación por capas y la evidencia (código, pantallas y pruebas).

CU-S2-01 · Administrar cuentas y roles

CU-S2-01 · Administrar cuentas y roles	
Historia de usuario	**HU-10.1** · Como administrador del sistema, quiero dar de alta, modificar y desactivar cuentas y asignar roles sincronizados con el directorio institucional, para que cada persona acceda solo a los módulos que su función requiere (RN-19).
Actor principal	Administrador del sistema (ACT-09)
Actor de apoyo	Directorio institucional (ACT-06), por ahora mediante un adaptador provisional
Origen	SCU-10 · proceso transversal · RN-19 · prioridad Debe · tamaño M
Tarjetas	S2-04 (servidor) · S2-05 (interfaz) · S2-06 (pruebas)
Precondiciones	Los roles existen en la base de datos (migración `4d9e0a2b7c6f_seed_identity_roles`). El administrador está en la pantalla Usuarios.
Flujo principal (alta)	1. El administrador pulsa «Nuevo usuario» y captura la cuenta institucional, el rol y, opcionalmente, la carga máxima. 2. El sistema valida el formato de los datos (esquema `UserCreate`). 3. El sistema verifica que la cuenta no esté ya registrada en SIGPI. 4. El sistema consulta el directorio institucional y obtiene nombre, correo y tipo de usuario. 5. El sistema crea la cuenta activa, le asigna el rol y la devuelve (HTTP 201). 6. La pantalla cierra el formulario y recarga el listado y los indicadores.
Flujos alternos	• **A1. Cuenta no reconocida por el directorio** (paso 4): HTTP 404; no se crea ningún registro y el formulario conserva lo capturado. • **A2. Cuenta ya registrada** (paso 3): HTTP 409. • **A3. Rol no configurado**: HTTP 422. • **A4. Datos inválidos** (rol inexistente, cuenta vacía, carga negativa, campos extra): HTTP 422. • **Modificar**: «Editar» cambia el rol o la carga máxima (`PATCH /usuarios/{id}`). • **Desactivar**: baja lógica (`DELETE /usuarios/{id}`); se conserva el historial.
Postcondiciones	La cuenta queda registrada con datos del directorio y un único rol vigente.
Endpoints	`GET /usuarios` · `POST /usuarios` · `PATCH /usuarios/{user_id}` · `DELETE /usuarios/{user_id}`

Criterios de aceptación

Criterio	Enunciado (Dado–Cuando–Entonces)	Estado y evidencia
AC1	Dado que ingreso una cuenta institucional válida, cuando asigno un rol compatible con su perfil, entonces el sistema registra la cuenta, notifica al usuario y lo asienta en la bitácora de auditoría.	Cumplido: registro `test_alta_valida_*` y prueba de pantalla Por integrar: auditoría el servicio existe (CU-S2-05); falta invocarlo desde el alta Pendiente: aviso servicio de notificaciones (S4)
AC2	Dado que el directorio no reconoce la cuenta, cuando intento darla de alta, entonces el sistema rechaza el alta.	Cumplido `test_cuenta_no_reconocida_*` (HTTP 404, sin registro) y prueba de pantalla

Tabla 6. Criterios de aceptación de HU-10.1.

Implementación

El servicio aplica las validaciones en orden: primero descarta duplicados, después consulta el directorio y, por último, verifica que el rol esté configurado. Cada condición lanza una excepción de negocio propia, que el router traduce a un código HTTP.

backend/app/services/usuario_service.py — líneas 29–64

```python
    async def create_user(
        self,
        *,
        institutional_id: str,
        role: RoleName,
        unit_id: uuid.UUID | None = None,
        max_load: int | None = None,
    ) -> User:
        existing = await self.repository.get_by_institutional_id(institutional_id)
        if existing is not None:
            raise DuplicateUserError(f"La cuenta {institutional_id} ya está registrada en SIGPI.")

        profile = await self.directory.fetch_profile(institutional_id)
        if profile is None:
            raise UnknownDirectoryAccountError(
                f"El directorio institucional no reconoce la cuenta {institutional_id}."
            )

        role_row = await self.repository.get_role_by_name(role)
        if role_row is None:
            raise RoleNotConfiguredError(f"El rol {role.value} no está configurado.")

        user = User(
            institutional_id=profile.institutional_id,
            full_name=profile.full_name,
            email=profile.email,
            user_type=profile.user_type,
            unit_id=unit_id,
            max_load=max_load,
            is_active=True,
        )
        created = await self.repository.add(user)
        await self.repository.assign_role(created.id, role_row.id)
        loaded = await self.repository.get_by_id(created.id)
        assert loaded is not None
        return loaded
```

Figura 5. Caso de uso de alta de cuentas sincronizado con el directorio (S2-04).

backend/app/api/usuarios.py — líneas 35–53

```python
@router.post("", response_model=UserRead, status_code=status.HTTP_201_CREATED)
async def create_user(
    payload: UserCreate, service: UsuarioService = Depends(get_usuario_service)
) -> UserRead:
    try:
        return await service.create_user(
            institutional_id=payload.institutional_id,
            role=payload.role,
            unit_id=payload.unit_id,
            max_load=payload.max_load,
        )
    except UnknownDirectoryAccountError as exc:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail=str(exc)) from exc
    except DuplicateUserError as exc:
        raise HTTPException(status_code=status.HTTP_409_CONFLICT, detail=str(exc)) from exc
    except RoleNotConfiguredError as exc:
        raise HTTPException(
            status_code=status.HTTP_422_UNPROCESSABLE_ENTITY, detail=str(exc)
        ) from exc
```

Figura 6. Router de usuarios: traducción de las excepciones de negocio a códigos HTTP (S2-04).

Pantalla de administración de usuarios

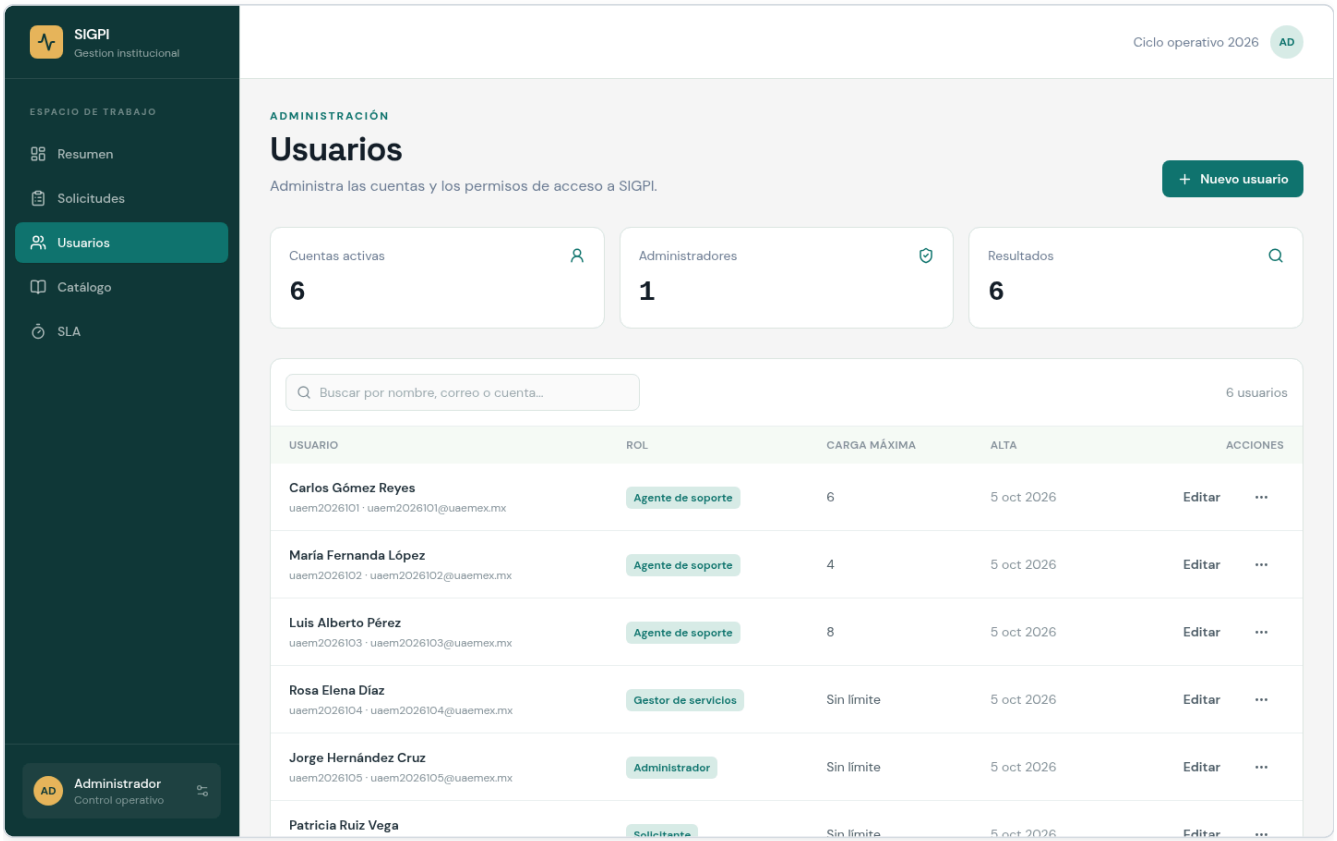


Figura 7. Pantalla Usuarios (S2-05): indicadores, búsqueda y listado de cuentas activas con su rol y carga máxima. Captura con datos de demostración.

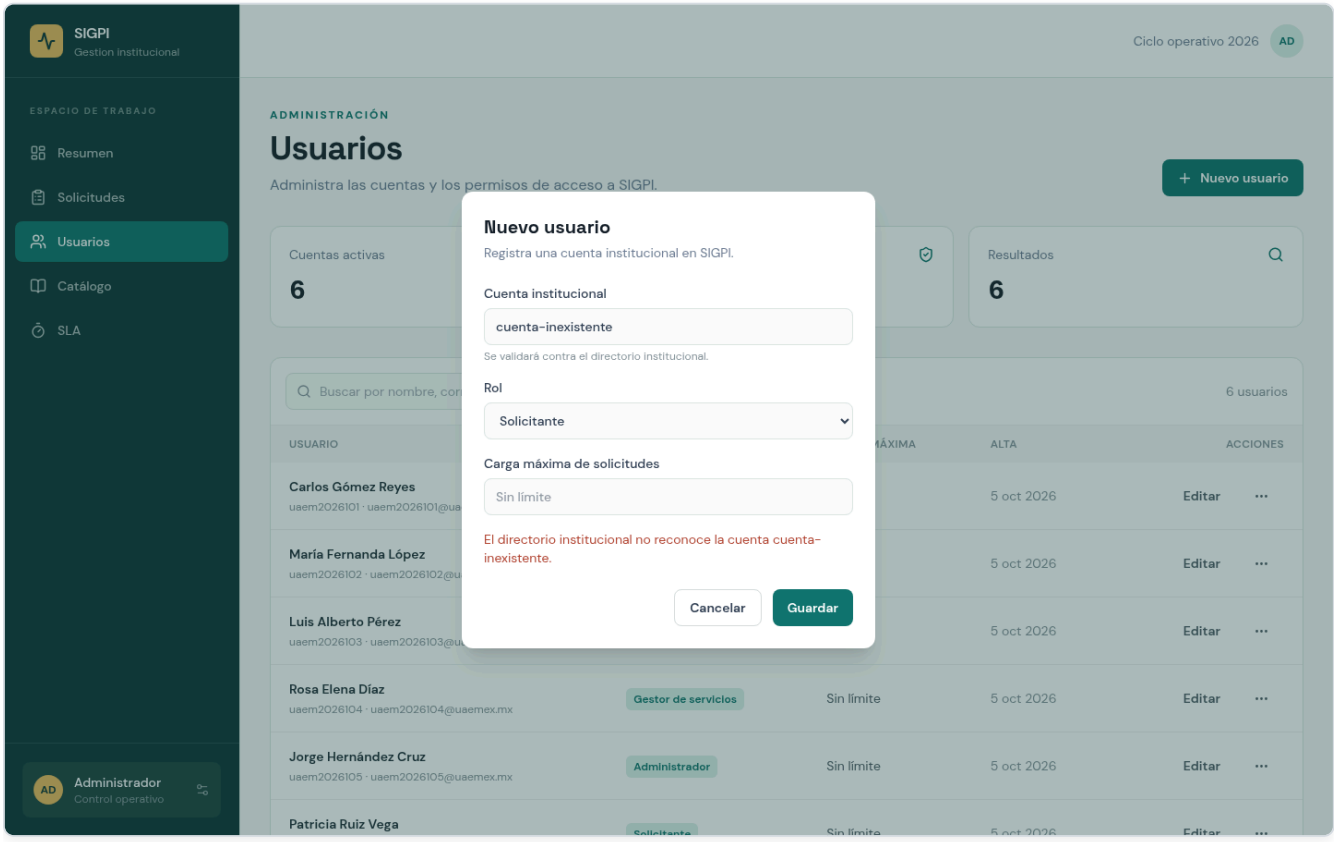


Figura 8. Flujo alterno A1: al registrar una cuenta que el directorio no reconoce, el formulario muestra el mensaje del servidor y no crea la cuenta.

Pruebas

Las pruebas de S2-06 cubren las dos capas. En el backend usan el router, el servicio y el repositorio reales contra SQLite en memoria (11 casos); en el frontend, Vitest y Testing Library verifican que la pantalla envíe el alta, recargue la lista y muestre el mensaje del directorio (5 casos).

backend/tests/api/test_usuarios.py — líneas 81–100

```python
def test_cuenta_no_reconocida_por_el_directorio_responde_404(client) -> None:
    response = _alta(client, institutional_id="cuenta-inexistente")

    assert response.status_code == 404
    assert "cuenta-inexistente" in response.json()["detail"]


def test_cuenta_no_reconocida_no_crea_ningun_usuario(client) -> None:
    _alta(client, institutional_id="cuenta-inexistente")

    assert client.get("/usuarios").json() == []


def test_cuenta_ya_registrada_responde_409(client) -> None:
    _alta(client)

    response = _alta(client)

    assert response.status_code == 409
    assert KNOWN_ACCOUNT in response.json()["detail"]
```

Figura 9. *Pruebas del flujo alterno «cuenta no reconocida» y del duplicado (S2-06).*

CU-S2-02 · Mantener el catálogo de técnicos y áreas

CU-S2-02 · Mantener el catálogo de técnicos y áreas	
Historia de usuario	**HU-11.1** · Como administrador del sistema, quiero mantener actualizadas las áreas de soporte, las especialidades, los turnos y las cargas máximas de los técnicos, para que la asignación automática use siempre información vigente (RN-08).
Actor principal	Administrador del sistema (ACT-09)
Origen	SCU-11 · proceso P-02 · RN-08 · prioridad Debe · tamaño M · depende de HU-10.1
Tarjetas	S2-10 (servidor) · S2-11 (interfaz) · S2-12 (pruebas)
Precondiciones	Existe al menos una unidad organizacional activa y los técnicos tienen cuenta activa (CU-S2-01).
Flujo principal	1. El administrador abre Catálogo → «Áreas y especialidades» y registra un área de servicio dentro de una unidad organizacional. 2. Registra las categorías del área y las especialidades de cada categoría. 3. Registra los turnos con hora de inicio y de fin. 4. En la pestaña «Técnicos», pulsa «Configurar» y define nivel de soporte, turno, carga máxima y especialidades habilitadas. 5. El sistema valida que las referencias existan y guarda los cambios. 6. El cambio queda disponible en la siguiente consulta, sin reiniciar el servicio.
Flujos alternos	• **A1. Referencia inexistente**: categoría sin área, especialidad sin categoría, turno o especialidad desconocidos → HTTP 422 con el detalle. • **A2. Turno inválido**: la hora de fin es anterior o igual a la de inicio → HTTP 422. • **A3. Actualización parcial**: los campos no enviados conservan su valor.
Postcondiciones	El catálogo vigente queda disponible para el motor de asignación (HU-03.1); un técnico solo puede recibir asignaciones de sus especialidades habilitadas (RN-08).
Endpoints	`GET /catalog/organizational-units` · `GET|POST /catalog/areas` · `/catalog/categories` · `/catalog/specialties` · `/catalog/shifts` · `GET /catalog/technicians?specialty_id=` · `PATCH /catalog/technicians/{user_id}`

Criterios de aceptación

Criterio	Enunciado	Estado y evidencia
AC1	Dado que doy de alta un área y asocio a sus técnicos, cuando defino especialidad, turno, categorías y carga máxima, entonces el sistema valida el catálogo y lo pone a disposición del motor sin reiniciar el servicio.	Cumplido 5 pruebas de API de vigencia inmediata (S2-12) y 13 pruebas del servicio (S2-10)

Tabla 7. Criterio de aceptación de HU-11.1.

Implementación

La vigencia inmediata se logra por diseño: el servicio no guarda nada en caché, así que cada consulta llega a la base de datos. La regla RN-08 se aplica en un único punto, la actualización del perfil del técnico, que verifica que todas las especialidades existan antes de asignarlas.

backend/app/services/catalog_service.py líneas 1–9

```python
"""Casos de uso del catálogo de técnicos y áreas de soporte (HU-11.1).

Reglas aplicadas:
- RN-08: un técnico solo puede recibir asignaciones de las especialidades que
  tiene habilitadas; este servicio es la única puerta para declararlas.
- Vigencia inmediata: no hay caché en esta capa; cada consulta llega a la base
  de datos, así que un cambio de turno o de carga máxima aplica en la siguiente
  asignación sin reiniciar el servicio.
"""
```

Figura 10. *Reglas que aplica el servicio del catálogo (S2-10).*

backend/app/services/catalog_service.py líneas 98–125

```python
    async def update_technician_profile(
        self, user_id: uuid.UUID, payload: TechnicianProfileUpdate
    ) -> User:
        technician = await self._technicians.get(user_id)
        if technician is None:
            raise CatalogValidationError(f"technician {user_id} does not exist")

        changes = payload.model_dump(exclude_unset=True)

        shift_id = changes.get("shift_id")
        if shift_id is not None and await self._shifts.get(shift_id) is None:
            raise CatalogValidationError(f"shift {shift_id} does not exist")

        if "specialty_ids" in changes and changes["specialty_ids"] is not None:
            wanted = list(dict.fromkeys(changes["specialty_ids"]))
            found = await self._specialties.get_many(wanted)
            missing = set(wanted) - {s.id for s in found}
            if missing:
                raise CatalogValidationError(
                    "unknown specialties: " + ", ".join(sorted(str(m) for m in missing))
                )
            technician.specialties = found

        for field in ("support_level", "shift_id", "max_load"):
            if field in changes:
                setattr(technician, field, changes[field])

        return await self._technicians.save(technician)
```

Figura 11. *Actualización del perfil del técnico con validación de referencias y RN-08 (S2-10).*

Pantalla del catálogo

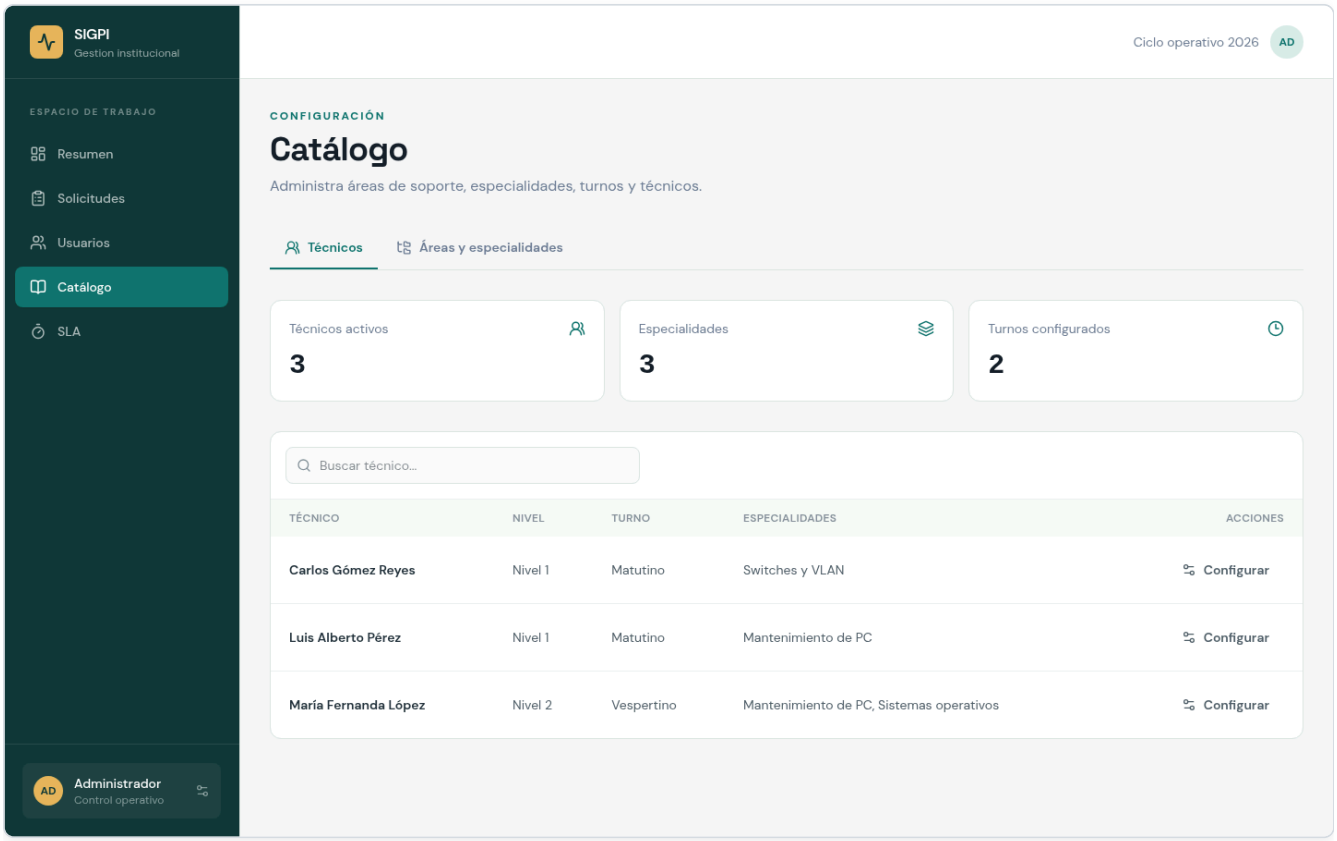


Figura 12. *Pantalla Catálogo, pestaña Técnicos (S2-11): nivel, turno y especialidades habilitadas de cada técnico.*

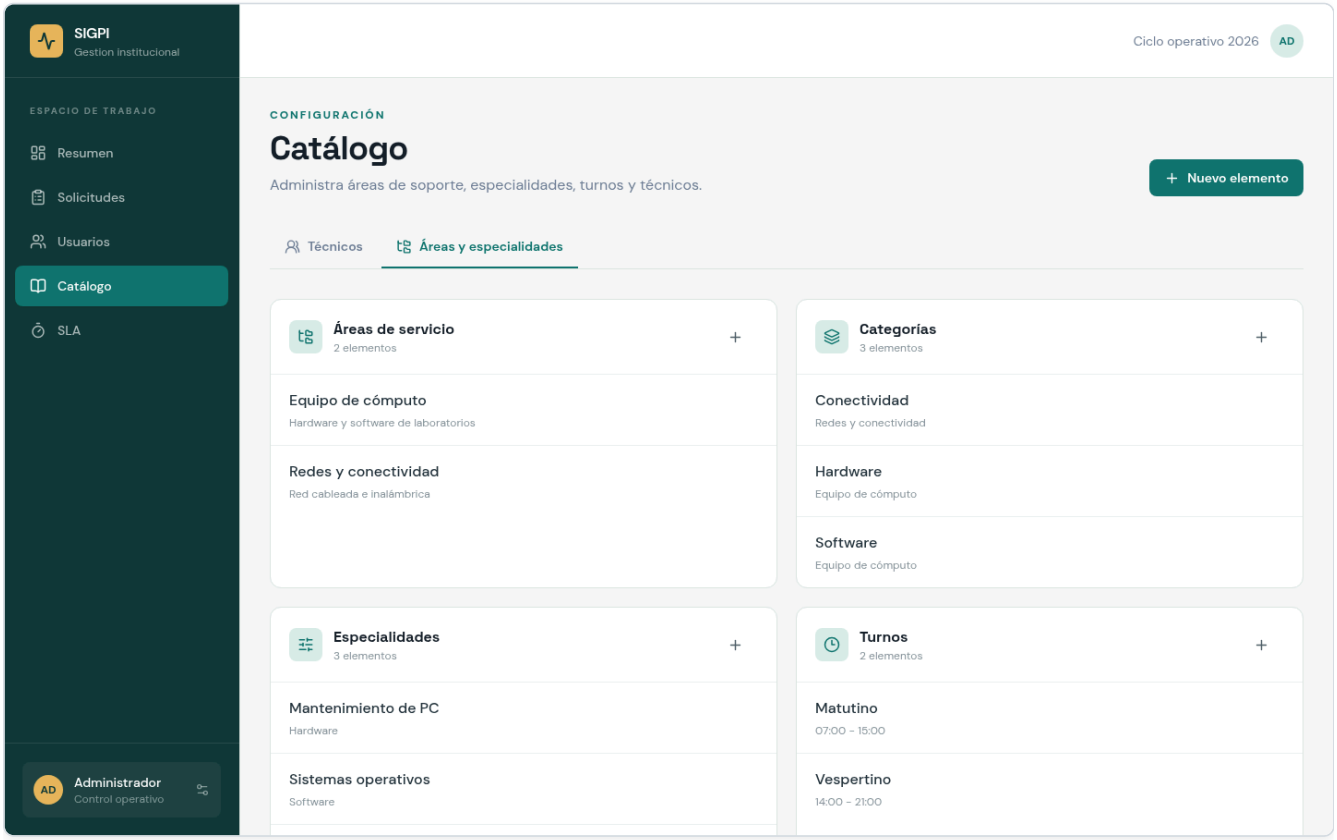


Figura 13. *Pantalla Catálogo, pestaña Áreas y especialidades (S2-11): áreas de servicio, categorías, especialidades y turnos.*

Pruebas

La prueba siguiente demuestra el criterio de vigencia inmediata: antes de la configuración, el filtro por especialidad no devuelve técnicos; después de asignarla, el mismo técnico aparece en la siguiente consulta sin reiniciar la aplicación.

backend/tests/api/test_catalogos.py líneas 153–169

```python
def test_filtro_por_especialidad_usa_la_configuracion_vigente(catalog_client) -> None:
    client, unit_id, technician_id = catalog_client
    _area_id, _category_id, specialty_id, _shift_id = _create_catalog(client, unit_id)

    before = client.get(f"/catalog/technicians?specialty_id={specialty_id}")
    assert before.status_code == 200
    assert before.json() == []

    response = client.patch(
        f"/catalog/technicians/{technician_id}",
        json={"specialty_ids": [specialty_id]},
    )
    assert response.status_code == 200, response.text

    after = client.get(f"/catalog/technicians?specialty_id={specialty_id}")
    assert after.status_code == 200
    assert [technician["id"] for technician in after.json()] == [str(technician_id)]
```

Figura 14. *Prueba de vigencia inmediata del catálogo (S2-12).*

CU-S2-03 · Registrar una solicitud con folio

CU-S2-03 · Registrar una solicitud con folio (servicio de alta)	
Historia de usuario	**HU-01.1** · Como usuario solicitante, quiero registrar una solicitud describiendo el problema y su ubicación física con mis credenciales institucionales, para obtener un folio de seguimiento y una fecha compromiso sin depender de canales informales.
Actor principal	Usuario solicitante (ACT-01)
Origen	SCU-01 · proceso P-01 · RN-01, RN-02, RN-03 · prioridad Debe · tamaño M · depende de HU-10.1
Tarjetas	S2-16 (servidor). La historia continúa en S3-02, S3-03 y S3-04 (pantalla del portal y pruebas de aceptación).
Precondiciones	El solicitante tiene una cuenta activa (CU-S2-01) y, si indica categoría, esta existe en el catálogo (CU-S2-02).
Flujo principal	1. El solicitante envía título, descripción, ubicación física y, opcionalmente, categoría, urgencia e impacto (canal «portal» por defecto). 2. El sistema valida el formato con el esquema `RequestCreate`. 3. El sistema verifica que el solicitante exista y esté activo (RN-02). 4. Si se indicó categoría, verifica que exista y esté activa (RN-03). 5. El sistema genera el siguiente folio consecutivo con formato `SIG-0001`. 6. El sistema registra la solicitud con estatus «Registrada» (`registered`) y la devuelve con HTTP 201.
Flujos alternos	• **A1. Solicitante inexistente o inactivo** → HTTP 422, no se genera folio. • **A2. Categoría inexistente o inactiva** → HTTP 422. • **A3. Solicitud relacionada inexistente** → HTTP 422. • **A4. Campos obligatorios vacíos** (título, descripción o ubicación) → HTTP 422, no se genera folio.
Postcondiciones	La solicitud queda registrada con un folio único que el usuario usará para darle seguimiento.
Endpoint	`POST /solicitudes`

Diagrama de secuencia

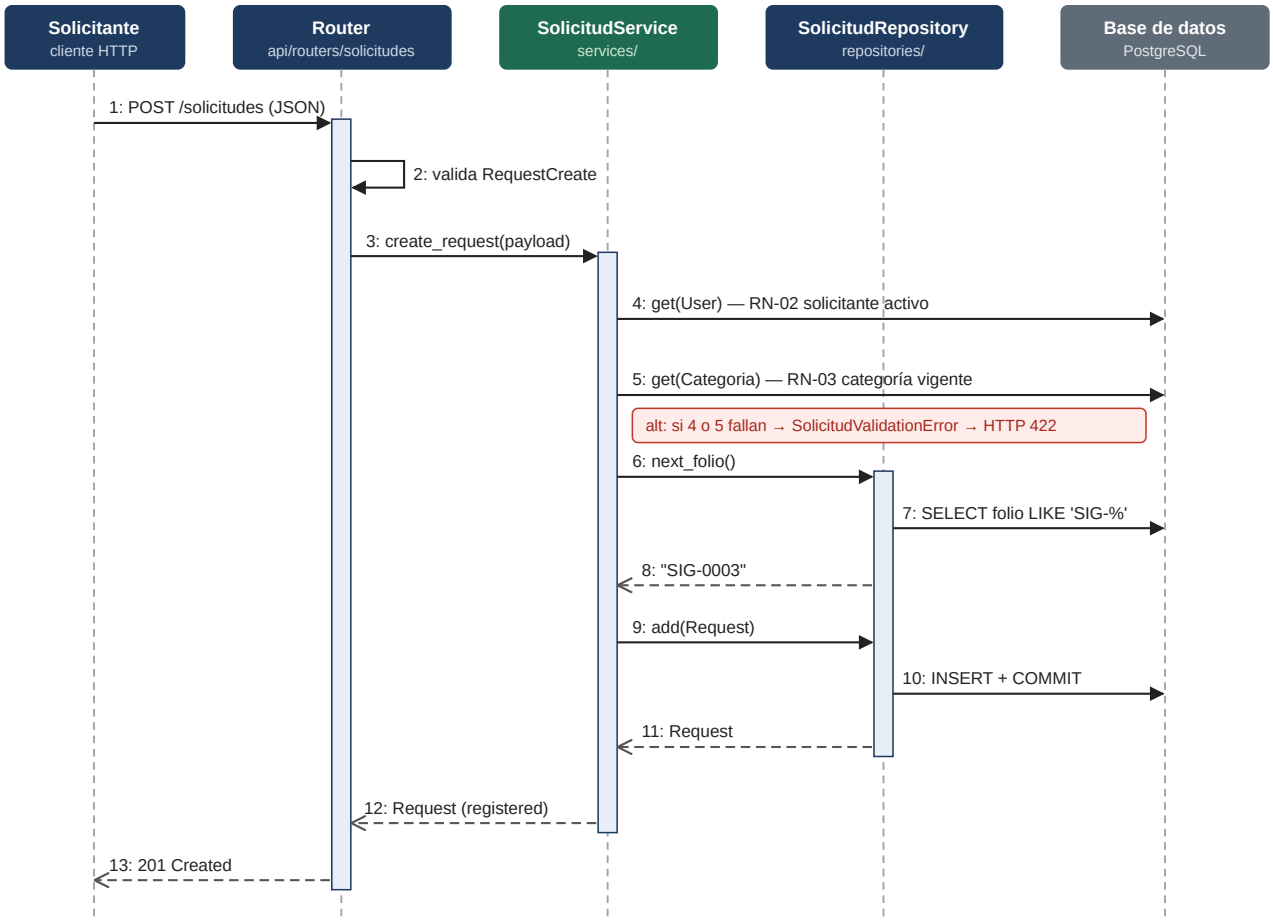


Figura 15. *Diagrama de secuencia del alta de una solicitud a través de las capas (S2-16). Elaboración propia.*

Criterios de aceptación

Criterio	Enunciado	Estado y evidencia
AC1	Dado que soy un usuario activo y autenticado, cuando capturo título, descripción de al menos 30 caracteres y ubicación y envío la solicitud, entonces el sistema genera un folio único y consecutivo, fija el estatus «Registrada», muestra el acuse y dispara la clasificación.	**Cumplido: folio y estatus** `test_alta_asigna_folio_*`, `test_folio_es_consecutivo_*` **Pendiente: acuse en el portal** S3-02 **Pendiente: clasificación** HU-02.1 (S4)
AC2	Dado que la descripción tiene menos de 30 caracteres o la ubicación está vacía, cuando intento enviar la solicitud, entonces el sistema rechaza el envío, señala el campo incompleto y no genera folio.	**Cumplido: campos vacíos** `test_alta_rechaza_datos_*` **Ajuste: mínimo de 30 caracteres** el esquema exige hoy al menos 1 carácter (sección 10)
AC3	Dado que mis credenciales son inválidas, cuando intento autenticarme tres veces seguidas, entonces el sistema ofrece la recuperación de contraseña.	**Fuera del alcance de S2** depende de la autenticación con el directorio (S3-03)

Tabla 8. Criterios de aceptación de HU-01.1 y su avance en el Sprint 2. La fecha compromiso que menciona la historia se obtiene del acuerdo vigente (CU-S2-06); su asociación al alta queda para el Sprint 3.

Implementación

backend/app/services/solicitud_service.py — líneas 22–57

```python
    async def create_request(self, payload: RequestCreate) -> Request:
        requester = await self._session.get(User, payload.requester_id)
        if requester is None or not requester.is_active:
            raise SolicitudValidationError(
                f"requester {payload.requester_id} does not exist or is inactive"
            )

        if payload.category_id is not None:
            category = await self._session.get(Categoria, payload.category_id)
            if category is None or not category.is_active:
                raise SolicitudValidationError(
                    f"category {payload.category_id} does not exist or is inactive"
                )

        if payload.related_request_id is not None:
            related = await self._session.get(Request, payload.related_request_id)
            if related is None:
                raise SolicitudValidationError(
                    f"related request {payload.related_request_id} does not exist"
                )

        request = Request(
            requester_id=payload.requester_id,
            category_id=payload.category_id,
            asset_id=payload.asset_id,
            related_request_id=payload.related_request_id,
            folio=await self._repository.next_folio(),
            title=payload.title,
            description=payload.description,
            physical_location=payload.physical_location,
            channel=payload.channel,
            status=RequestStatus.REGISTERED,
            urgency=payload.urgency,
            impact=payload.impact,
        )
        return await self._repository.add(request)
```

Figura 16. *Servicio de alta de solicitudes con las validaciones RN-02 y RN-03 (S2-16).*

El folio se calcula en el repositorio a partir del mayor folio existente con el patrón `SIG-n`; la columna `folio` tiene restricción de unicidad en el modelo, por lo que la base de datos impide duplicados.

backend/app/repositories/solicitud_repository.py — líneas 17–26

```python
    async def next_folio(self) -> str:
        result = await self._session.scalars(
            select(Request.folio).where(Request.folio.like("SIG-%")).order_by(Request.folio.desc())
        )
        highest = 0
        for folio in result:
            match = _FOLIO_PATTERN.fullmatch(folio)
            if match:
                highest = max(highest, int(match.group(1)))
        return f"SIG-{highest + 1:04d}"
```

Figura 17. *Generación del folio consecutivo en la capa de acceso a datos (S2-16).*

Pruebas

backend/tests/api/test_solicitudes.py — líneas 86–104

```python
def test_folio_es_consecutivo_y_persiste_entre_altas(request_client) -> None:
    client, requester_id, _category_id = request_client

    first = client.post("/solicitudes", json=_payload(requester_id, title="Primera solicitud"))
    second = client.post("/solicitudes", json=_payload(requester_id, title="Segunda solicitud"))

    assert first.status_code == 201, first.text
    assert second.status_code == 201, second.text
    assert first.json()["folio"] == "SIG-0001"
    assert second.json()["folio"] == "SIG-0002"


def test_rn02_rechaza_solicitante_inexistente_o_inactivo(request_client) -> None:
    client, _requester_id, _category_id = request_client

    response = client.post("/solicitudes", json=_payload(uuid.uuid4()))

    assert response.status_code == 422
    assert "requester" in response.json()["detail"]
```

Figura 18. *Pruebas del folio consecutivo y del rechazo de un solicitante inexistente (S2-16).*

CU-S2-04 · Registrar el avance en la bitácora (diseño)

En este sprint la historia HU-04.1 solo comprometía la tarjeta de **diseño** S2-17: definir la entidad de la bitácora y sus contratos. El servicio y la pantalla se construyen en el Sprint 3 (S3-11, S3-12 y S3-13).

CU-S2-04 · Registrar el avance en una bitácora única	
Historia de usuario	**HU-04.1** · Como técnico de soporte, quiero registrar el diagnóstico, la solución aplicada y los cambios de estatus en una bitácora única, para dejar constancia y dar visibilidad del avance al usuario y a mi jefe sin llamadas de seguimiento.
Actor principal	Técnico de soporte (ACT-02 · ACT-03)
Origen	SCU-04 · proceso P-02 · RN-10 · prioridad Debe · tamaño M
Tarjeta	S2-17 (diseño)
Decisiones de diseño	• Cada entrada guarda **marca de tiempo, autor, estatus anterior y estatus nuevo**, como exige el criterio AC2. • La bitácora solo admite **altas**: no existe un contrato de actualización, de modo que el historial no se reescribe. • Los estatus usan el mismo enumerado `RequestStatus` que la solicitud, lo que evita valores inconsistentes.
Criterio cubierto por el diseño	**AC2.** Dado que cambio el estatus de una solicitud, cuando guardo el cambio, entonces la bitácora registra marca de tiempo, autor y estatus anterior y nuevo.

backend/app/models/request.py — líneas 69–82

```python
class RequestLog(Base):
    __tablename__ = "request_logs"

    id: Mapped[uuid.UUID] = mapped_column(Uuid, primary_key=True, default=uuid.uuid4)
    request_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("service_requests.id"), index=True)
    author_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("users.id"))
    previous_status: Mapped[RequestStatus | None] = mapped_column(String(30))
    new_status: Mapped[RequestStatus | None] = mapped_column(String(30))
    action: Mapped[str] = mapped_column(String(120))
    notes: Mapped[str | None] = mapped_column(Text)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=datetime.utcnow)

    request: Mapped[Request] = relationship(back_populates="logs")
    author: Mapped["User"] = relationship()
```

Figura 19. Entidad RequestLog de la bitácora de atención (modelo).

backend/app/schemas/request.py — líneas 84–109

```python
class RequestLogCreate(BaseModel):
    """Fields accepted when appending a new entry to a request's bitácora."""

    model_config = ConfigDict(extra="forbid")

    request_id: uuid.UUID
    author_id: uuid.UUID
    previous_status: RequestStatus | None = None
    new_status: RequestStatus | None = None
    action: str = Field(min_length=1, max_length=120)
    notes: str | None = None


class RequestLogRead(BaseModel):
    """Public representation of a bitácora entry returned by the API."""

    model_config = ConfigDict(from_attributes=True)

    id: uuid.UUID
    request_id: uuid.UUID
    author_id: uuid.UUID
    previous_status: RequestStatus | None
    new_status: RequestStatus | None
    action: str
    notes: str | None
    created_at: datetime
```

Figura 20. *Contratos de alta y lectura de la bitácora (S2-17).*

CU-S2-05 · Auditar las acciones de administración

CU-S2-05 · Auditar las acciones de administración	
Historia de usuario	**HU-10.3** · Como administrador del sistema, quiero que toda acción de administración quede registrada con fecha, autor y cambio, para contar con evidencia auditable (RN-19).
Actor principal	Administrador del sistema (ACT-09)
Origen	SCU-10 · RN-19 · prioridad Debe · tamaño S · depende de HU-10.1
Tarjetas	S2-07 (diseño: modelo inalterable) · S2-08 (servidor: servicio de auditoría) · S2-09 (pruebas de inalterabilidad)
Precondiciones	La migración `5e1f7a9c3b2d_make_audit_log_immutable` está aplicada en la base de datos.
Flujo principal	1. El administrador guarda un cambio de permisos, roles o catálogos. 2. El caso de uso que realiza el cambio llama a `AuditoriaService.registrar_cambio` con el autor, la acción, la entidad afectada y los valores anterior y nuevo. 3. El sistema agrega una entrada a `audit_logs` con la fecha del cambio y, si se conoce, la dirección IP de origen. 4. Para revisar la evidencia, el sistema devuelve el historial de una entidad o de un autor en orden cronológico.
Flujos alternos	• **A1. Intento de modificar una entrada**: el disparador `audit_logs_immutable` rechaza el `UPDATE` con el error «audit_logs is append-only» (código 55006) y el valor original se conserva. • **A2. Intento de eliminar una entrada**: el disparador rechaza el `DELETE` y el registro permanece.
Postcondiciones	La entrada de auditoría queda guardada de forma permanente; ningún usuario ni proceso de la aplicación puede alterarla.
Interfaz	Servicio interno de la capa de aplicación: no expone endpoint ni pantalla propios; lo consumen los demás casos de uso de administración.

Criterios de aceptación

Criterio	Enunciado	Estado y evidencia
AC1	Dado que cambio permisos, roles o catálogos, cuando guardo el cambio, entonces el sistema registra autor, fecha y valores anterior y nuevo en una bitácora que no puede modificarse.	Cumplido: registro de autor, fecha y valores `test_registra_autor_fecha_*` Cumplido: no puede modificarse `test_auditoria_rechaza_update_*`, `*_delete_*` Por integrar invocar el servicio desde cuentas, roles y catálogo (sección 10)

Tabla 9. Criterio de aceptación de HU-10.3.

Implementación

La inalterabilidad no depende de la disciplina del código: la garantiza la propia base de datos. La migración crea una función y un disparador de PostgreSQL que se ejecutan antes de cualquier `UPDATE` o `DELETE` sobre la tabla y cancelan la operación. Además se agregan índices para consultar el historial por fecha y por entidad.

backend/alembic/versions/5e1f7a9c3b2d_make_audit_log_immutable.py líneas 13–45

```python
def upgrade() -> None:
    op.create_index(
        "ix_audit_logs_created_at",
        "audit_logs",
        ["created_at"],
    )
    op.create_index(
        "ix_audit_logs_entity_lookup",
        "audit_logs",
        ["entity_type", "entity_id", "created_at"],
    )
    op.execute(
        """
        CREATE OR REPLACE FUNCTION prevent_audit_log_mutation()
        RETURNS trigger
        LANGUAGE plpgsql
        AS $$
        BEGIN
            RAISE EXCEPTION 'audit_logs is append-only; % is not allowed',
                TG_OP
                USING ERRCODE = '55006';
        END;
        $$;
        """
    )
    op.execute(
        """
        CREATE TRIGGER audit_logs_immutable
        BEFORE UPDATE OR DELETE ON audit_logs
        FOR EACH ROW
        EXECUTE FUNCTION prevent_audit_log_mutation();
        """
    )
```

Figura 21. *Migración que vuelve inalterable la tabla de auditoría mediante un disparador (S2-07).*

El servicio solo ofrece operaciones de alta y de consulta; el repositorio no tiene métodos para actualizar ni eliminar entradas.

backend/app/services/auditoria_service.py — líneas 11–40

```python
class AuditoriaService:
    """Records immutable before/after snapshots for a business change."""

    def __init__(self, repository: AuditoriaRepository) -> None:
        self._repository = repository

    async def registrar_cambio(
        self,
        *,
        actor_id: uuid.UUID | None,
        action: str,
        entity_type: str,
        entity_id: str | uuid.UUID,
        old_values: dict[str, Any] | None = None,
        new_values: dict[str, Any] | None = None,
        ip_address: str | None = None,
        created_at: datetime | None = None,
    ) -> Auditoria:
        entry = Auditoria(
            actor_id=actor_id,
            action=action,
            entity_type=entity_type,
            entity_id=str(entity_id),
            old_values=old_values,
            new_values=new_values,
            ip_address=ip_address,
        )
        if created_at is not None:
            entry.created_at = created_at
        return await self._repository.add(entry)
```

Figura 22. *Servicio de auditoría: registro del autor, la entidad y los valores anterior y nuevo (S2-08).*

Pruebas

Las pruebas de inalterabilidad se ejecutan contra PostgreSQL, porque el disparador es propio de ese motor. Cada prueba inserta una entrada, intenta alterarla dentro de un *savepoint*, comprueba que la base de datos lo impide y verifica que el valor original sigue intacto; al final toda la transacción se revierte, por lo que no deja datos en la base.

backend/tests/services/test_auditoria.py — líneas 48–66

```python
@pytest.mark.asyncio
async def test_auditoria_rechaza_update_y_conserva_valores(audit_connection) -> None:
    async with audit_connection.begin():
        entry_id = await _insert_audit_entry(audit_connection)
        savepoint = await audit_connection.begin_nested()

        with pytest.raises(DBAPIError, match="append-only"):
            await audit_connection.execute(
                update(Auditoria)
                .where(Auditoria.id == entry_id)
                .values(new_values={"value": "altered"})
            )

        await savepoint.rollback()
        row = await audit_connection.execute(
            select(Auditoria.new_values).where(Auditoria.id == entry_id)
        )

        assert row.scalar_one() == {"value": "original"}
```

Figura 23. *Prueba de inalterabilidad: la base de datos rechaza el UPDATE y conserva el valor original (S2-09).*

CU-S2-06 · Definir y versionar los acuerdos de nivel de servicio

CU-S2-06 · Definir y versionar los acuerdos de nivel de servicio (SLA)	
Historia de usuario	**HU-08.1** · Como jefe del Departamento de TIC, quiero definir y versionar los tiempos de primera respuesta y de resolución por categoría y prioridad, para formalizar compromisos verificables frente a la comunidad y ajustarlos con evidencia.
Actor principal	Jefe del Departamento de TIC (ACT-04)
Origen	SCU-08 · procesos P-01 y P-03 · RN-05 · prioridad Debe · tamaño M · depende de HU-10.1
Tarjetas	S2-13 (servidor) · S2-14 (interfaz) · S2-15 (pruebas). El modelo versionado se diseñó en S1-07.
Precondiciones	Existen las categorías del catálogo (CU-S2-02).
Flujo principal	1. El jefe abre la pantalla SLA y crea un acuerdo para una categoría, o el acuerdo predeterminado (sin categoría). 2. Pulsa «Nueva versión», indica la fecha de entrada en vigor y los minutos de primera respuesta y de resolución para una o varias prioridades (P1 a P4). 3. El sistema valida que la resolución sea mayor que la primera respuesta y que la fecha sea posterior a la de la versión vigente. 4. El sistema cierra la versión vigente de cada prioridad modificada (fecha de término = nueva fecha de entrada en vigor) y guarda la versión nueva con el siguiente número. 5. La pantalla muestra el histórico de versiones con su estado: «Vigente» o «Cerrada el …».
Flujo de consulta	1. El jefe elige una categoría y una prioridad y pulsa «Consultar» en «Consultar acuerdo vigente». 2. El sistema devuelve la versión vigente del acuerdo de la categoría en la fecha indicada. 3. Si la categoría no tiene acuerdo para esa prioridad, aplica el predeterminado y devuelve un aviso de la omisión.
Flujos alternos	• **A1. Inconsistencia de tiempos**: resolución menor o igual a la primera respuesta → HTTP 422 y no se guarda ninguna versión. • **A2. Fecha no posterior** a la de la versión vigente → HTTP 422. • **A3. Acuerdo duplicado**: segunda categoría con acuerdo activo o segundo predeterminado → HTTP 409. • **A4. Acuerdo inexistente** → HTTP 404. • **A5. Sin acuerdo propio ni predeterminado** para la prioridad → HTTP 404.
Postcondiciones	Existe una versión vigente por prioridad y el histórico completo; una solicitud conserva el acuerdo que estaba vigente cuando se registró.
Endpoints	`GET|POST /sla/agreements` · `GET|POST /sla/agreements/{id}/versions` · `GET /sla/effective?priority=&category_id=&at=`

Criterios de aceptación

Criterio	Enunciado	Estado y evidencia
AC1	Dado que defino tiempos para una combinación de categoría y prioridad, cuando confirmo la fecha de entrada en vigor, entonces el sistema registra la nueva versión, conserva el histórico y las solicitudes en curso mantienen el acuerdo con el que se registraron.	Cumplido `test_version_nueva_conserva_el_historico_*`, `test_solicitud_en_curso_conserva_*` y prueba de pantalla «AC1»
AC2	Dado que el tiempo de resolución es menor o igual al de primera respuesta, cuando intento guardar, entonces el sistema lo rechaza señalando la inconsistencia.	Cumplido validación en esquema, servicio y pantalla; `test_resolucion_menor_o_igual_*`, `test_inconsistencia_no_guarda_*` y 3 pruebas de pantalla «AC2»

AC3	Dado que se registra una solicitud en una combinación sin acuerdo configurado, cuando el sistema procesa la clasificación, entonces aplica el acuerdo predeterminado y me notifica la omisión.	**Cumplido: predeterminado y aviso** `test_combinacion_sin_acuerdo_*` y prueba de pantalla «AC3» **Pendiente: invocarlo al clasificar** HU-02.1 (S4)

Tabla 10. Criterios de aceptación de HU-08.1.

Implementación

Publicar una versión nunca modifica los tiempos de una versión anterior: solo le asigna la fecha de término. Por eso el histórico es completo y la consulta del acuerdo vigente puede hacerse en cualquier fecha pasada, que es lo que permite que una solicitud en curso mantenga su compromiso original.

backend/app/services/sla_service.py — líneas 87–127

```python
    async def publish_version(
        self, agreement_id: uuid.UUID, payload: SlaVersionCreate
    ) -> list[ServiceLevelVersion]:
        agreement = await self._repository.get_agreement(agreement_id)
        if agreement is None:
            raise SlaNotFoundError(f"No existe el acuerdo {agreement_id}.")
        if not agreement.is_active:
            raise SlaValidationError("El acuerdo está inactivo y no admite versiones nuevas.")

        valid_from = as_utc(payload.valid_from)
        closing: list[ServiceLevelVersion] = []
        for times in payload.times:
            if times.resolution_minutes <= times.first_response_minutes:
                raise SlaValidationError(
                    f"Inconsistencia en {times.priority.value}: el tiempo de resolución debe "
                    "ser mayor que el de primera respuesta."
                )
            current = await self._repository.open_version(agreement_id, times.priority.value)
            if current is not None:
                if as_utc(current.valid_from) >= valid_from:
                    raise SlaValidationError(
                        f"La fecha de entrada en vigor debe ser posterior a la de la versión "
                        f"{current.version} de {times.priority.value}."
                    )
                closing.append(current)

        next_number = await self._repository.latest_version_number(agreement_id) + 1
        for current in closing:
            current.valid_to = valid_from
        new_versions = [
            ServiceLevelVersion(
                agreement_id=agreement_id,
                version=next_number,
                priority=times.priority.value,
                first_response_minutes=times.first_response_minutes,
                resolution_minutes=times.resolution_minutes,
                valid_from=valid_from,
            )
            for times in payload.times
        ]
        return await self._repository.save_versions(new_versions)
```

Figura 24. *Publicación de una versión: validación de la inconsistencia (AC2) y cierre de la versión anterior conservando el histórico (AC1) (S2-13).*

backend/app/services/sla_service.py líneas 139–166

```python
    async def resolve_effective(
        self,
        category_id: uuid.UUID | None,
        priority: Priority,
        at: datetime | None = None,
    ) -> EffectiveSla:
        moment = as_utc(at) if at is not None else datetime.now(UTC)

        if category_id is not None:
            own = await self._repository.find_active_for_category(category_id)
            if own is not None:
                version = await self._repository.find_effective(own.id, priority.value, moment)
                if version is not None:
                    return self._build(own, version, used_default=False, notice=None)

        default = await self._repository.find_default()
        if default is not None:
            version = await self._repository.find_effective(default.id, priority.value, moment)
            if version is not None:
                notice = (
                    f"No hay acuerdo configurado para la categoría y la prioridad "
                    f"{priority.value}; se aplicó el acuerdo predeterminado «{default.name}»."
                )
                return self._build(default, version, used_default=True, notice=notice)

        raise SlaNotConfiguredError(
            f"No hay acuerdo propio ni predeterminado para la prioridad {priority.value}."
        )
```

Figura 25. *Resolución del acuerdo vigente con respaldo en el acuerdo predeterminado y aviso de la omisión (AC3) (S2-13).*

La regla de consistencia se valida también en el esquema de entrada, de modo que la API rechaza la petición antes de llegar al servicio:

backend/app/schemas/sla.py líneas 38–54

```python
class SlaPriorityTimes(BaseModel):
    """Tiempos comprometidos para una prioridad, en minutos."""

    model_config = ConfigDict(extra="forbid")

    priority: Priority
    first_response_minutes: int = Field(gt=0)
    resolution_minutes: int = Field(gt=0)

    @model_validator(mode="after")
    def _resolution_after_first_response(self) -> "SlaPriorityTimes":
        if self.resolution_minutes <= self.first_response_minutes:
            raise ValueError(
                f"Inconsistencia en {self.priority.value}: el tiempo de resolución debe ser "
                "mayor que el de primera respuesta."
            )
        return self
```

Figura 26. *Validación de la inconsistencia de tiempos en el esquema Pydantic (S2-13).*

Pantalla de configuración de SLA

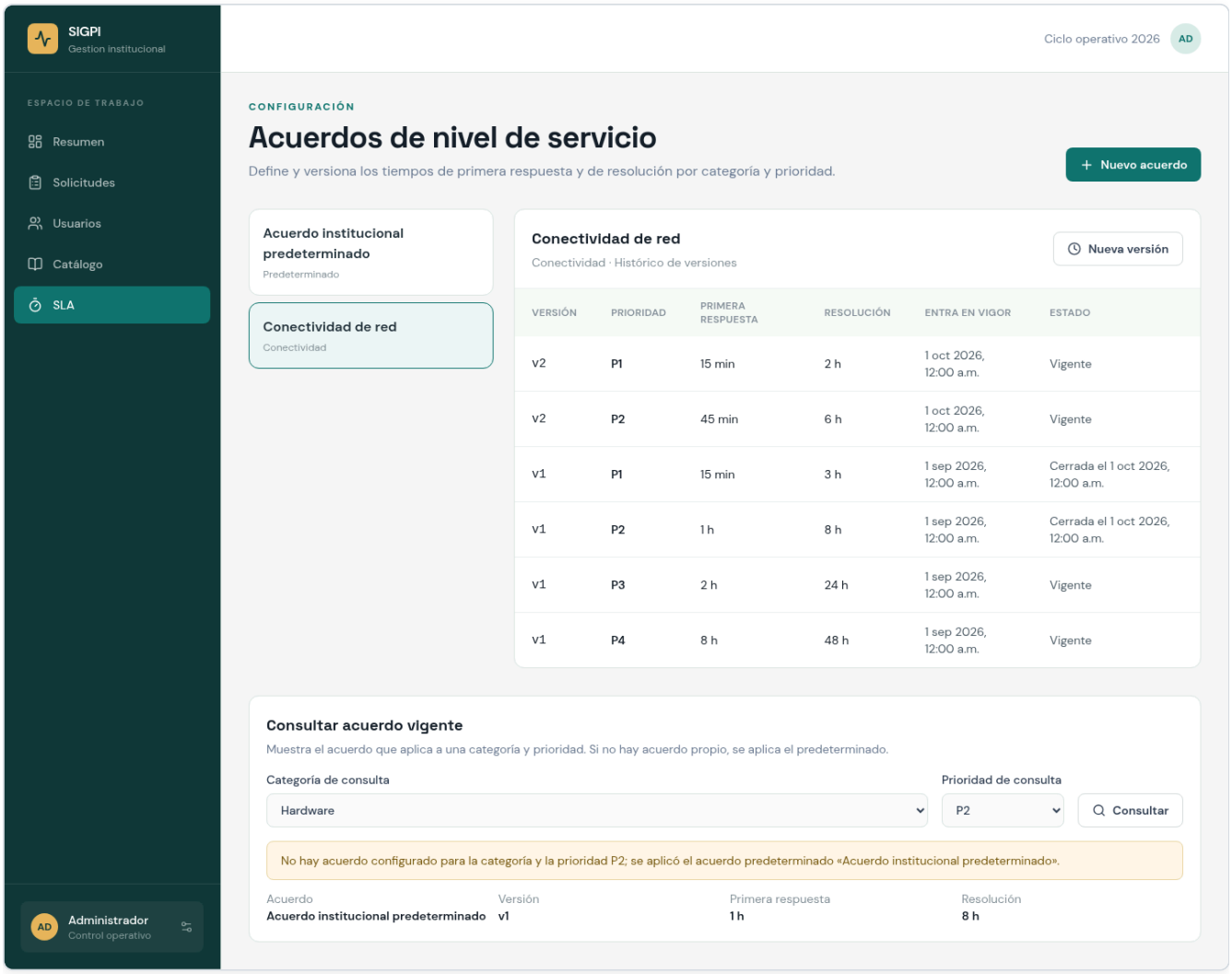


Figura 27. *Pantalla SLA (S2-14): histórico del acuerdo «Conectividad de red» con la versión 2 vigente para P1 y P2 y la versión 1 cerrada el 1 de octubre (AC1). Abajo, la consulta de una categoría sin acuerdo propio aplica el predeterminado y muestra el aviso (AC3).*

Pruebas

S2-15 aporta 22 pruebas del servicio con SQLite en memoria, y S2-14, 11 pruebas de la pantalla con Vitest. La prueba siguiente demuestra la parte central del AC1: tras publicar la versión 2, una consulta con la fecha de una solicitud anterior sigue devolviendo la versión 1.

backend/tests/services/test_sla_service.py líneas 147–176

```python
@pytest.mark.asyncio
async def test_version_nueva_conserva_el_historico_y_cierra_la_anterior(
    service, category_id
) -> None:
    agreement = await _agreement(service, category_id)
    await service.publish_version(agreement.id, _version(T0, _times(Priority.P1, 15, 120)))

    await service.publish_version(agreement.id, _version(T1, _times(Priority.P1, 10, 60)))

    history = await service.list_versions(agreement.id, priority=Priority.P1)
    assert [v.version for v in history] == [2, 1]
    assert history[0].valid_to is None
    assert history[1].valid_to is not None
    assert history[1].first_response_minutes == 15
    assert history[1].resolution_minutes == 120


@pytest.mark.asyncio
async def test_solicitud_en_curso_conserva_el_acuerdo_con_el_que_se_registro(
    service, category_id
) -> None:
    agreement = await _agreement(service, category_id)
    await service.publish_version(agreement.id, _version(T0, _times(Priority.P1, 15, 120)))
    await service.publish_version(agreement.id, _version(T1, _times(Priority.P1, 10, 60)))

    antes = await service.resolve_effective(category_id, Priority.P1, T0 + timedelta(days=5))
    despues = await service.resolve_effective(category_id, Priority.P1, T1 + timedelta(days=1))

    assert (antes.version, antes.resolution_minutes) == (1, 120)
    assert (despues.version, despues.resolution_minutes) == (2, 60)
```

Figura 28. *Pruebas del histórico y de la solicitud en curso que conserva su acuerdo (S2-15).*

backend/tests/services/test_sla_service.py líneas 239–253

```python
async def test_combinacion_sin_acuerdo_aplica_el_predeterminado_y_avisa(
    service, category_id
) -> None:
    default = await _agreement(service)
    await service.publish_version(default.id, _version(T0, _times(Priority.P3, 60, 480)))
    own = await _agreement(service, category_id)
    await service.publish_version(own.id, _version(T0, _times(Priority.P1, 15, 120)))

    result = await service.resolve_effective(category_id, Priority.P3, T0 + timedelta(days=1))

    assert result.used_default is True
    assert result.agreement_id == default.id
    assert result.resolution_minutes == 480
    assert result.notice is not None
    assert "P3" in result.notice
```

Figura 29. *Prueba del acuerdo predeterminado con aviso de la omisión (AC3, S2-15).*

6. Base ejecutable: migraciones, integración continua y contenedores

6.1 Modelo relacional y migraciones (S2-01)

Migración / semilla	Contenido
`744530feb03d_initial_migration`	Esquema inicial de las entidades del núcleo (Sprint 1).
`2a4f6b8c9d10_versioned_sla_constraints`	Restricciones del SLA versionado (S1-07), base de CU-S2-06.
`3c7e8f1a2b4d_add_user_shift`	Relación del técnico con su turno, requerida por el catálogo (HU-11.1).
`4d9e0a2b7c6f_seed_identity_roles`	Semilla idempotente de los siete roles del sistema.
`5e1f7a9c3b2d_make_audit_log_immutable`	Índices de consulta y disparador que vuelve inalterable la auditoría (S2-07).
`python -m app.seeds all`	Comandos para cargar unidades organizacionales, categorías y el catálogo de prioridades (pruebas en `tests/test_seeds.py`).

Tabla 11. Migraciones y semillas vigentes al cierre del Sprint 2.

6.2 Integración continua y Docker Compose (S2-02)

Cada *pull request* y cada *push* a `main` ejecutan dos trabajos: el backend (ruff y pytest) y el frontend (ESLint, Prettier, Vitest, compilación TypeScript y build de Vite). Docker Compose levanta PostgreSQL 16, la API y el frontend con un solo comando. El estado del pipeline al corte se describe en la sección 10.

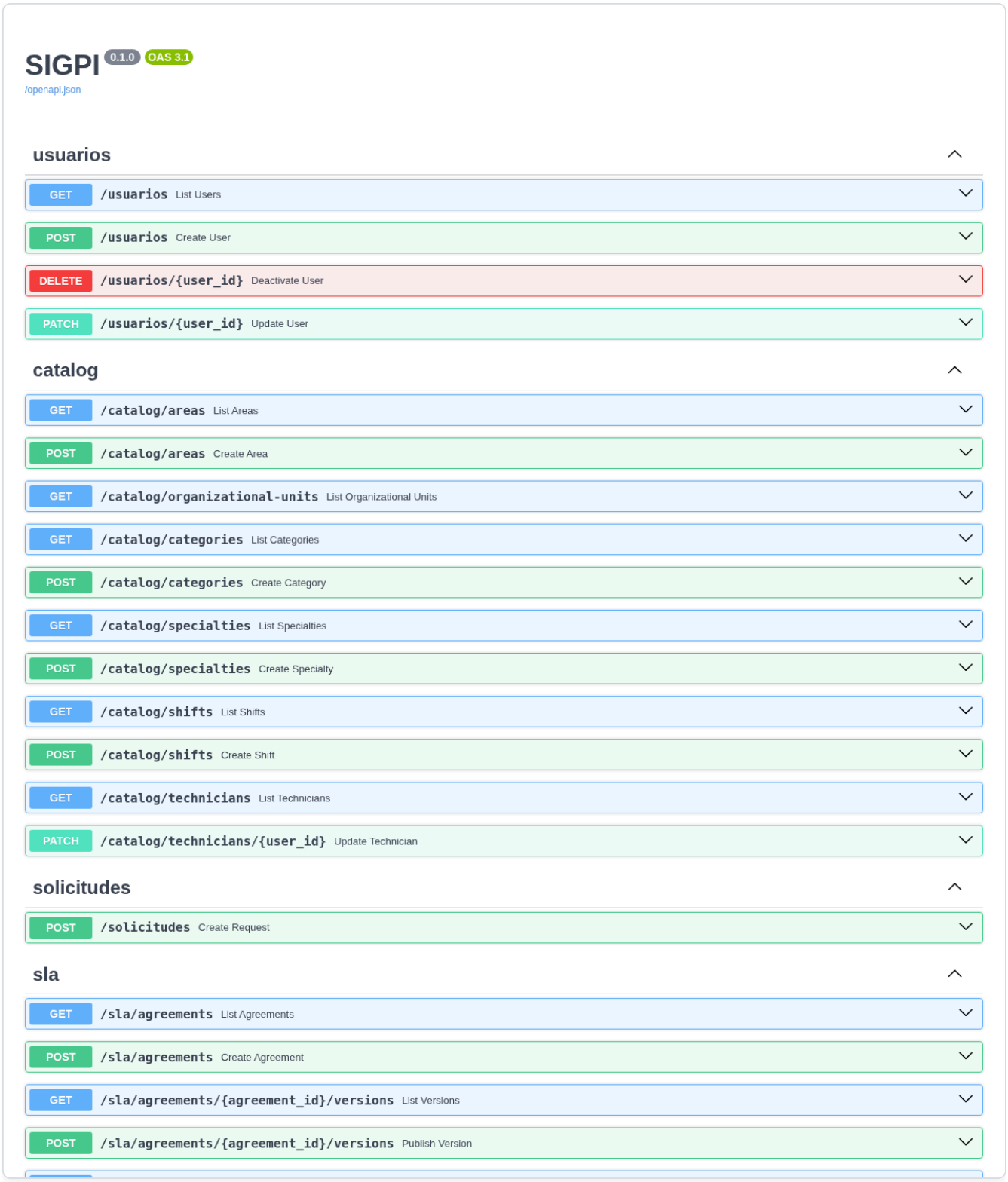


Figura 30. *Documentación OpenAPI generada por FastAPI con los endpoints entregados en el Sprint 2: usuarios, catálogo, solicitudes y SLA. Es la fuente de la que se generan los tipos del frontend (S2-03).*

.github/workflows/ci.yml líneas 9–55

```yaml
jobs:
  backend:
    name: Backend (ruff + pytest)
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: backend
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.14"
          cache: "pip"
          cache-dependency-path: backend/requirements.txt
      - name: Instalar dependencias
        run: pip install -r requirements.txt
      - name: ruff (estilo)
        run: ruff check app tests
      - name: pytest
        run: pytest -q

  frontend:
    name: Frontend (eslint + tsc + build)
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: frontend
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v4
        with:
          version: 10
      - uses: actions/setup-node@v4
        with:
          node-version: "22"
          cache: "pnpm"
          cache-dependency-path: frontend/pnpm-lock.yaml
      - name: Instalar dependencias
        run: pnpm install --frozen-lockfile
      - name: ESLint
        run: pnpm run lint
      - name: Formato (Prettier)
        run: pnpm run format:check
      - name: Pruebas (Vitest)
        run: pnpm run test
      - name: Compilar (tsc + vite build)
        run: pnpm run build
```

Figura 31. *Trabajos del pipeline de integración continua en GitHub Actions (S2-02).*

7. Pruebas y calidad

Al corte del sprint, el backend tiene **95 pruebas y todas pasan en la ejecución local con PostgreSQL** (en el pipeline de GitHub, que no lo tiene, pasan 93 y las 2 de inalterabilidad marcan error; ver la observación 4) y el frontend **16 pruebas que también pasan**; ruff no reporta observaciones en el código Python. Las pruebas de API y las de los servicios de auditoría y SLA usan las capas reales con una base SQLite en memoria; las del servicio del catálogo (S2-10) usan repositorios en memoria; las dos de inalterabilidad usan PostgreSQL porque prueban un disparador de ese motor.

Archivo	Tarjeta	Historia	Casos	Qué verifica
`tests/api/test_usuarios.py`	S2-06	HU-10.1	11	Alta válida, cuenta no reconocida (404), duplicado (409), rol no configurado y datos inválidos (422)
`features/users/users-page.test.tsx`	S2-06	HU-10.1	5	Listado, estado vacío, alta válida y mensaje del directorio en la pantalla
`tests/test_catalog_service.py`	S2-10	HU-11.1	13	Reglas del servicio del catálogo y RN-08
`tests/api/test_catalogos.py`	S2-12	HU-11.1	5	Vigencia inmediata, filtro por especialidad, referencias no vigentes, unidades organizacionales
`tests/api/test_solicitudes.py`	S2-16	HU-01.1	8	Folio y estatus inicial, folio consecutivo, RN-02, RN-03 y campos obligatorios
`tests/services/test_auditoria_service.py`	S2-09	HU-10.3	2	Registro de autor, fecha y valores; orden del historial
`tests/services/test_auditoria.py`	S2-09	HU-10.3	2	La base de datos rechaza UPDATE y DELETE (PostgreSQL)
`tests/services/test_sla_service.py`	S2-15	HU-08.1	22	Acuerdos únicos, versiones e histórico, inconsistencias, predeterminado y fechas
`features/sla/sla-page.test.tsx`	S2-14	HU-08.1	11	Histórico, AC1, AC2 y AC3 en la pantalla
`tests/test_seeds.py`	S2-01	—	3	Semillas idempotentes
Pruebas heredadas del Sprint 1 (prioridad RN-05, permisos, esquemas, SLA)			29	Siguen pasando: el sprint no introdujo regresiones
Total			**111**	**95 de backend + 16 de frontend · 0 fallidas con PostgreSQL local**

Tabla 12. Pruebas automatizadas por tarjeta. Ejecución local del 4 de octubre de 2026, con PostgreSQL disponible; sin él, 2 pruebas de inalterabilidad marcan error (observación 4).

```
terminal

$ pytest tests/api -v
tests/api/test_catalogos.py::test_unidades_organizacionales_activas_se_listan_para_crear_areas PASSED
tests/api/test_catalogos.py::test_catalogo_creado_aparece_inmediatamente_en_listados PASSED
tests/api/test_catalogos.py::test_cambio_de_perfil_tecnico_es_visible_en_la_siguiente_consulta PASSED
tests/api/test_catalogos.py::test_filtro_por_especialidad_usa_la_configuracion_vigente PASSED
tests/api/test_catalogos.py::test_actualizacion_rechaza_referencias_no_vigentes PASSED
tests/api/test_solicitudes.py::test_alta_asigna_folio_y_estado_inicial PASSED
tests/api/test_solicitudes.py::test_folio_es_consecutivo_y_persiste_entre_altas PASSED
tests/api/test_solicitudes.py::test_rn02_rechaza_solicitante_inexistente_o_inactivo PASSED
tests/api/test_solicitudes.py::test_rn02_rechaza_solicitante_inactivo PASSED
tests/api/test_solicitudes.py::test_rn03_rechaza_categoria_inexistente PASSED
tests/api/test_solicitudes.py::test_alta_rechaza_datos_obligatorios_invalidos[title] PASSED
tests/api/test_solicitudes.py::test_alta_rechaza_datos_obligatorios_invalidos[description] PASSED
tests/api/test_solicitudes.py::test_alta_rechaza_datos_obligatorios_invalidos[physical_location] PASSED
tests/api/test_usuarios.py::test_alta_valida_crea_la_cuenta_con_los_datos_del_directorio PASSED
tests/api/test_usuarios.py::test_alta_valida_aparece_en_el_listado_de_cuentas_activas PASSED
tests/api/test_usuarios.py::test_cuenta_no_reconocida_por_el_directorio_responde_404 PASSED
tests/api/test_usuarios.py::test_cuenta_no_reconocida_no_crea_ningun_usuario PASSED
tests/api/test_usuarios.py::test_cuenta_ya_registrada_responde_409 PASSED
tests/api/test_usuarios.py::test_rol_no_configurado_responde_422 PASSED
tests/api/test_usuarios.py::test_datos_de_alta_invalidos_responden_422[rol_invalido] PASSED
tests/api/test_usuarios.py::test_datos_de_alta_invalidos_responden_422[id_vacio] PASSED
tests/api/test_usuarios.py::test_datos_de_alta_invalidos_responden_422[carga_negativa] PASSED
tests/api/test_usuarios.py::test_datos_de_alta_invalidos_responden_422[campo_extra] PASSED
tests/api/test_usuarios.py::test_datos_de_alta_invalidos_responden_422[sin_id] PASSED
============================== 24 passed in 2.18s ==============================
```

Figura 32. *Ejecución de las 24 pruebas de API (S2-06, S2-12 y S2-16).*

```
terminal

$ pytest tests/services -v
tests/services/test_auditoria.py::test_auditoria_rechaza_update_y_conserva_valores PASSED
tests/services/test_auditoria.py::test_auditoria_rechaza_delete_y_conserva_registro PASSED
tests/services/test_auditoria_service.py::test_registra_autor_fecha_y_valores_anterior_nuevo PASSED
tests/services/test_auditoria_service.py::test_historial_de_entidad_conserva_el_orden_de_los_cambios PASSED
tests/services/test_sla_service.py::test_crear_acuerdo_de_categoria PASSED
tests/services/test_sla_service.py::test_no_permite_dos_acuerdos_activos_para_la_misma_categoria PASSED
tests/services/test_sla_service.py::test_no_permite_dos_acuerdos_predeterminados PASSED
tests/services/test_sla_service.py::test_acuerdo_de_categoria_inexistente_se_rechaza PASSED
tests/services/test_sla_service.py::test_acuerdo_predeterminado_no_lleva_categoria PASSED
tests/services/test_sla_service.py::test_acuerdo_que_no_es_predeterminado_requiere_categoria PASSED
tests/services/test_sla_service.py::test_primera_version_queda_vigente_sin_fecha_de_termino PASSED
tests/services/test_sla_service.py::test_version_nueva_conserva_el_historico_y_cierra_la_anterior PASSED
tests/services/test_sla_service.py::test_solicitud_en_curso_conserva_el_acuerdo_con_el_que_se_registro PASSED
tests/services/test_sla_service.py::test_version_nueva_solo_cierra_las_prioridades_que_modifica PASSED
tests/services/test_sla_service.py::test_fecha_de_vigencia_no_puede_ser_anterior_a_la_version_vigente PASSED
tests/services/test_sla_service.py::test_version_de_acuerdo_inexistente_responde_no_encontrado PASSED
tests/services/test_sla_service.py::test_resolucion_menor_o_igual_a_primera_respuesta_se_rechaza[60-60] PASSED
tests/services/test_sla_service.py::test_resolucion_menor_o_igual_a_primera_respuesta_se_rechaza[60-30] PASSED
tests/services/test_sla_service.py::test_tiempos_no_positivos_se_rechazan PASSED
tests/services/test_sla_service.py::test_una_prioridad_no_puede_repetirse_en_la_misma_version PASSED
tests/services/test_sla_service.py::test_inconsistencia_no_guarda_ninguna_version PASSED
tests/services/test_sla_service.py::test_combinacion_sin_acuerdo_aplica_el_predeterminado_y_avisa PASSED
tests/services/test_sla_service.py::test_categoria_sin_acuerdo_propio_usa_el_predeterminado PASSED
tests/services/test_sla_service.py::test_sin_acuerdo_propio_ni_predeterminado_falla PASSED
tests/services/test_sla_service.py::test_antes_de_la_entrada_en_vigor_no_hay_acuerdo PASSED
tests/services/test_sla_service.py::test_fechas_sin_zona_horaria_se_interpretan_como_utc PASSED
============================== 26 passed in 1.38s ==============================
```

Figura 33. *Ejecución de las 26 pruebas de servicio de auditoría y SLA (S2-09 y S2-15).*

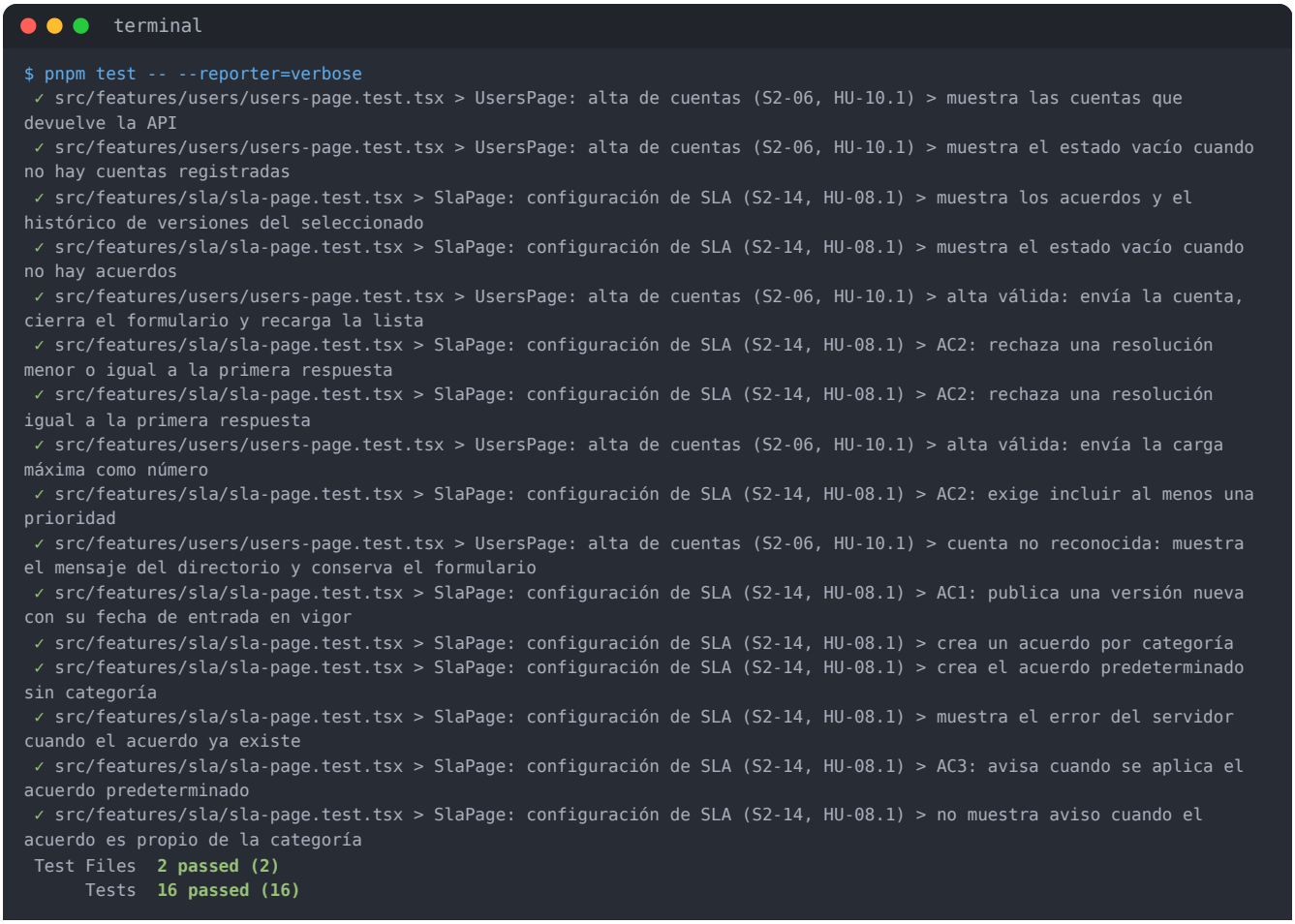


Figura 34. *Ejecución de las 16 pruebas de las pantallas Usuarios y SLA con Vitest (S2-06 y S2-14).*

8. Trazabilidad: historia → tarjeta → código → prueba

RUP exige que cada requisito pueda seguirse hasta su prueba. La tabla vincula cada historia del sprint con sus tarjetas, los archivos que la implementan y las pruebas que la verifican. Los commits llevan el identificador de la tarjeta (por ejemplo, `#S2-13` o `feat(S2-16)`), lo que permite reconstruir esta matriz desde el historial de Git.

Historia	Tarjeta	Entregable (código)	Prueba	Estado
HU-10.1	S2-04	`api/usuarios.py` · `services/usuario_service.py` · `repositories/usuario_repository.py` · `domain/directory.py`	`tests/api/test_usuarios.py` `users-page.test.tsx`	Realizado
	S2-05	`features/users/users-page.tsx` · `api/usuarios.ts`		Realizado
	S2-06	Pruebas de backend y de pantalla		Realizado
HU-11.1	S2-10	`api/routers/catalog.py` · `services/catalog_service.py` · `repositories/catalog_repository.py`	`tests/test_catalog_service.py`	Realizado
	S2-11	`features/catalog/catalog-page.tsx` · `api/catalog.ts`	`tests/api/test_catalogos.py`	Realizado
	S2-12	Pruebas de vigencia inmediata		Realizado
HU-01.1	S2-16	`api/routers/solicitudes.py` · `services/solicitud_service.py` · `repositories/solicitud_repository.py`	`tests/api/test_solicitudes.py`	Realizado
HU-04.1	S2-17	`schemas/request.py` (`RequestLogCreate`, `RequestLogRead`) · `models/request.py`	Se prueba con el servicio en S3-13	Realizado
HU-10.3	S2-07	`models/auditoria.py` · migración `5e1f7a9c3b2d`	`tests/services/test_auditoria.py` `tests/services/test_auditoria_service.py`	Realizado
	S2-08	`services/auditoria_service.py` · `repositories/auditoria_repository.py`		Realizado
	S2-09	Pruebas de inalterabilidad y del servicio		Realizado
HU-08.1	S2-13	`api/routers/sla.py` · `services/sla_service.py` · `repositories/sla_repository.py` · `schemas/sla.py`	`tests/services/test_sla_service.py` `sla-page.test.tsx`	Realizado
	S2-14	`features/sla/sla-page.tsx` · `use-sla.ts` · `api/sla.ts`		Realizado
	S2-15	Pruebas de versiones, inconsistencias y predeterminado		Realizado
Base	S2-01 S2-02 S2-03	`alembic/versions/*` · `app/seeds/` `.github/workflows/ci.yml` · `docker-compose.yml` · Dockerfiles `frontend/src/api/openapi.ts` · `types.ts` · `client.ts`	`tests/test_seeds.py` Pipeline CI Compilador TypeScript	Realizado

Tabla 13. Matriz de trazabilidad del Sprint 2.

9. Velocidad del sprint

Como el tablero solo registra horas planeadas, la velocidad se calcula con las horas planeadas de las tarjetas entregadas (Registro de velocidad del Sprint 2, sección 1).

Concepto	Comprometido	Entregado	Avance
Tarjetas de desarrollo	17	17	100 %
Horas de desarrollo	144 h	144 h	100 %
Historias con todas sus tarjetas del sprint	6	6	100 %
Tarjetas de gestión cerradas	4 (10 h)	1 (3 h)	30 % · 3 al cierre

Tabla 14. Velocidad comprometida frente a entregada.

Integrante	Asignadas	Ejecutadas	Tarjetas ejecutadas
Jesús Iniesta	45 h	77 h	S2-01, S2-03, S2-05, S2-07, S2-08, S2-09, S2-11, S2-12, S2-16
Emmanuel Bernal	34 h	51 h	S2-02, S2-06, S2-10, S2-13, S2-14, S2-15
Haide Sánchez	33 h	16 h	S2-04, S2-17
Laura Rodríguez	32 h	0 h	—
Total	**144 h**	**144 h**	

Tabla 15. Horas de desarrollo asignadas en el tablero y ejecutadas, por integrante. Fuente: Registro de velocidad del Sprint 2, tabla 3.

> **Lectura para la retrospectiva.** El sprint entregó el 100 % del desarrollo comprometido, pero el 34 % de las horas (49 h) las ejecutó una persona distinta a la asignada. El Registro de velocidad propone, para el Sprint 3, confirmar la capacidad real de cada integrante antes de la planeación, reasignar las tarjetas que lo requieran y empezar a registrar horas reales además de las planeadas.

10. Observaciones y ajustes para el Sprint 3

Todas las tarjetas de desarrollo del sprint están entregadas. Las observaciones siguientes no reabren tarjetas: son integraciones entre casos de uso y ajustes de calidad detectados durante la revisión, que se proponen como trabajo del Sprint 3.

#	Observación	Acción propuesta
1	El servicio de auditoría (CU-S2-05) todavía no se invoca desde los casos de uso de cuentas, roles y catálogo. Esto también deja pendiente el «asentar en la auditoría» del AC1 de HU-10.1.	Llamar a `AuditoriaService.registrar_cambio` desde `usuario_service` y `catalog_service`, con su prueba.
2	El alta de solicitudes aún no usa el acuerdo vigente, por lo que no calcula la fecha compromiso de HU-01.1; el modelo ya tiene el campo `service_level_version_id`.	Resolver el SLA vigente con `SlaService.resolve_effective` durante la clasificación (S3/S4).
3	El AC2 de HU-01.1 pide una descripción de al menos 30 caracteres; `RequestCreate` exige actualmente al menos 1.	Cambiar a `min_length=30` y agregar el caso a `test_solicitudes.py` (S3-04).
4	Pipeline de `main`: las dos pruebas de inalterabilidad requieren PostgreSQL y el trabajo de backend no lo tiene; Prettier marca 4 archivos del frontend (`openapi.ts`, `client.ts`, `sla.ts` y `catalog-page.tsx`).	Agregar un servicio PostgreSQL al trabajo de backend, excluir el archivo generado de Prettier y formatear los demás.
5	`openapi.ts` se generó una vez, pero el proyecto no tiene un script para regenerarlo.	Agregar `pnpm generate:api` con openapi-typescript.
6	El directorio institucional se simula con `StubDirectoryClient`.	Adaptador de Microsoft 365 en S3-03; el puerto del dominio ya está definido.
7	La pantalla Solicitudes muestra datos de ejemplo; el alta solo es accesible por la API.	Conectar el formulario del portal al endpoint en S3-02.
8	Tarjetas de gestión S2-18 y S2-21 en validación; S2-20 se cubre con este documento.	Reasignar el rol de Product Owner y registrar la validación del Sprint Backlog 2 y del Registro de velocidad.

Tabla 16. Observaciones y acciones propuestas para el siguiente sprint.

> **Conclusión.** El Sprint 2 cumplió su objetivo de Elaboración: dejó una base ejecutable con migraciones, contenedores, integración continua y un cliente tipado desde OpenAPI, y entregó en sus capas los seis casos de uso comprometidos. La administración de cuentas, el catálogo técnico y los acuerdos de nivel de servicio quedan completos con pantalla y pruebas; la auditoría queda garantizada por la propia base de datos; el alta de solicitudes genera folios consecutivos con sus validaciones, y la bitácora de atención tiene su diseño listo. El trabajo del Sprint 3 empieza por conectar estos casos de uso entre sí (auditoría en la administración y SLA en las solicitudes) y por dejar el pipeline en verde.

Referencias

Boehm, B., y Turner, R. (2003). *Balancing agility and discipline: A guide for the perplexed*. Addison-Wesley.

Cockburn, A. (2001). *Writing effective use cases*. Addison-Wesley.

Cohn, M. (2004). *User stories applied: For agile software development*. Addison-Wesley.

Kruchten, P. (2004). *The Rational Unified Process: An introduction* (3.ª ed.). Addison-Wesley.

Equipo SIGPI. (2026a). *Historias de usuario del Sistema SIGPI* (versión 3). Facultad de Ingeniería, UAEM.

Equipo SIGPI. (2026b). *Tablero Kanban del proyecto SIGPI* (versión 2). Facultad de Ingeniería, UAEM.

Equipo SIGPI. (2026c). *Sprint Backlog 2*; *RiskLog v2*; *Registro de velocidad del Sprint 2*. Facultad de Ingeniería, UAEM.

Schwaber, K., y Sutherland, J. (2020). *The Scrum Guide*. https://scrumguides.org